

Programação de Aplicativos

Prof. Carlos Roberto da Silva Filho

O que é uma linguagem de programação?

- É uma **linguagem formal** que funciona por meio de uma série de **instruções**, **símbolos**, **palavras-chave**, **regras semânticas e sintáticas**.
- De certo modo, pode-se dizer que é a forma pela qual o **hardware** (máquina) e o **programador** se “**comunicam**”.
- A linguagem de programação permite que um programador crie **programas** a partir de um **conjunto de ordens, ações consecutivas, dados e algoritmos**.

Tipos de Linguagem de Programação

- As linguagens de programação podem ser classificadas como **alto nível** e **baixo nível**.
- Também podem ser classificadas conforme o **paradigma de programação**.
- Um **paradigma de programação** é a **forma utilizada para resolver um problema** computacional.
- Linguagens podem suportar mais de um paradigma (linguagens multiparadigma).

Linguagem de Baixo nível

- A linguagem de **baixo nível** é mais próxima da linguagem de máquina, ou seja, do **hardware**.
- Essas linguagens têm o objetivo de se comunicar com o **hardware** de uma forma mais otimizada.
- A linguagem de máquina, conhecida como linguagem de primeira geração, é formada por códigos binários (0 e 1).
- Uma linguagem de programação de baixo nível, de segunda geração, muito usada é a linguagem **Assembly**.

Linguagem de Alto nível

- As linguagens de alto nível são aquelas cuja **sintaxe é voltada para o entendimento humano**, ou seja, do programador.
- Elas **abstraem conceitos** voltados para a máquina e **sintetizam em comandos**.
- Exemplos de linguagens de alto nível são: Java, C#, C, C++, Python, Javascript, PHP, Ruby, Visual Basic, Go (criada pelo google), Swift (criada pela Apple), R, Objective-C.

Linguagem de Alto nível

- As linguagens de alto nível podem ser **compiladas** ou **interpretadas**.
- **Compilador**: consiste no processo de **conversão do código fonte** para uma linguagem de máquina através do compilador, fazendo a análise e a síntese da linguagem.
- **Tradutor**: **interpreta** os programas escritos em uma linguagem de programação, traduzindo para a linguagem de máquina do computador.

Linguagens compiladas x interpretadas

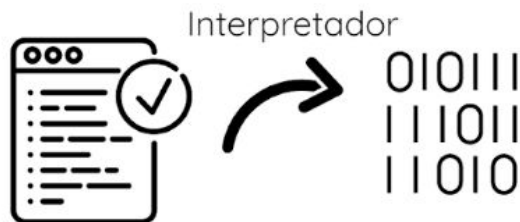
```
010111  
111011  
11010
```

Cliente já recebe o código
pronto para ser executado



Cliente

X



O código é interpretado e
convertido para o cliente e
então executado



Cliente

Ambiente de Desenvolvimento Integrado

- **IDE** ou **Integrated Development Environment** - Ambiente de Desenvolvimento Integrado, é um software que auxilia no desenvolvimento de aplicações.
- Muito utilizado por desenvolvedores, com o objetivo de facilitar diversos processos, pois combinam ferramentas comuns em uma única interface gráfica do usuário (GUI).

Ambiente de Desenvolvimento Integrado

- A IDE provê diversos benefícios, como a análise de todo o código a ser escrito, facilita identificação de “bugs” causados por erros de digitação, autocompleta trechos de códigos, ...
- **Características comuns** presentes em IDEs são:
- Editor de código-fonte, preenchimento inteligente de código, Compilador ou interpretador, debugger, geração automática de código, refatoração.

Estruturas de Controle de Programação

- Estruturas de controle são mecanismos de linguagens usados para especificar o fluxo de um programa.
- As estruturas típicas de controle de fluxo são:
- Estrutura condicional simples, composta e aninhada;
- Estrutura condicional múltipla escolha (case);
- Estrutura de Repetição: Enquanto, Faça-enquanto, Para-faça.

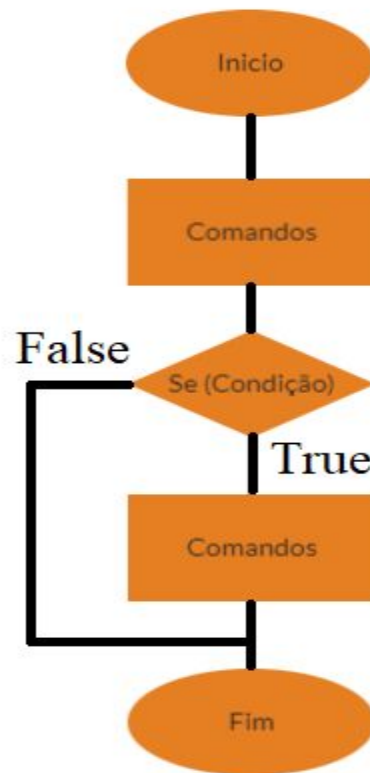
Estrutura condicional simples - IF

- Em JS serve para especificar um bloco de código que deverá ser executado, caso a condição seja verdadeira e não será executada, caso a condição seja falsa.

if (expressão de teste) {

Bloco de código a ser executado se a expressão for **True**

}



Estrutura condicional composta - IF/ELSE

- Em JS serve para especificar um bloco de código que deverá ser executado, caso a condição seja verdadeira e um segundo bloco de código, caso a condição seja falsa.

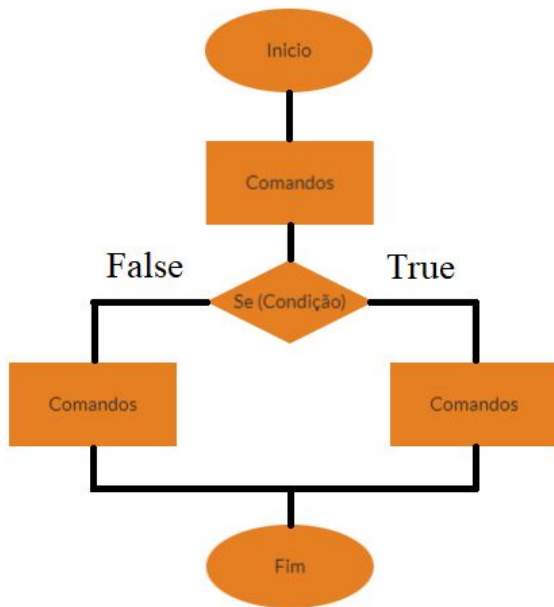
If (expressão de teste) {

Bloco de código a ser executado se a expressão for True

} **Else** {

Bloco de código a ser executado se a expressão for False

}



Estrutura condicional aninhada - IF/ELSE

- Em JS serve para especificar uma nova condição a ser testada.

If (expressão de teste) {

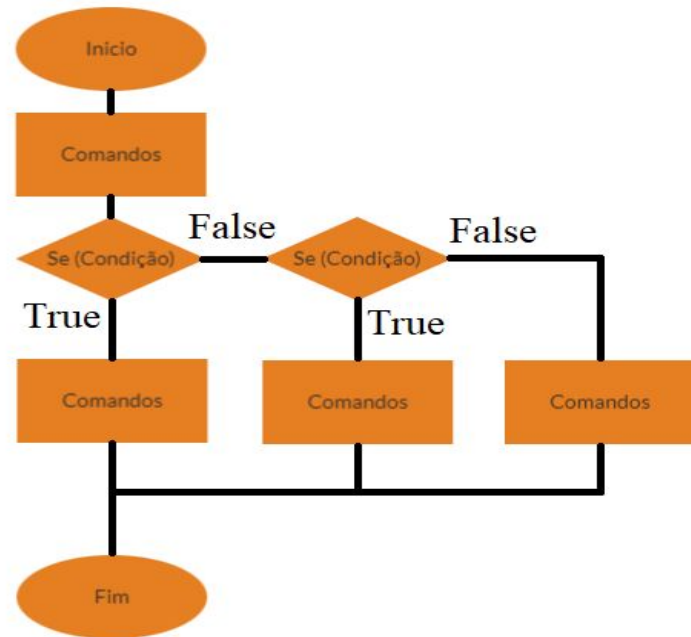
Bloco de código a ser executado se a expressão for True

} **Else If** (expressão de teste 2) {

Bloco de código a ser executado se a expressão for False e nova condição True

} **Else** {

Bloco de código a ser executado se a expressão for False
}



Estrutura condicional Switch / Case

- Em JS é empregada quando é preciso realizar diferentes ações, em função de várias condições.

switch (expressão de teste) {

case situação 1: Comandos a executar;

Break;

case situação 2: Comandos a executar;

Break;

default: Comandos a executar;

Break;

Estrutura For

- Em JS é empregada empregada quando é preciso realizar uma quantidade finita e conhecida de ações.

```
For (contador=1; contador<=condição; contador++) {
```

```
Comandos a executar;
```

```
}
```

Exemplo: Para realizar uma consulta em uma tabela de dados, onde deseja-se encontrar o maior número de itens de um determinado produto, pode-se realizar a consulta do item 1 até o item 500.

Estrutura While

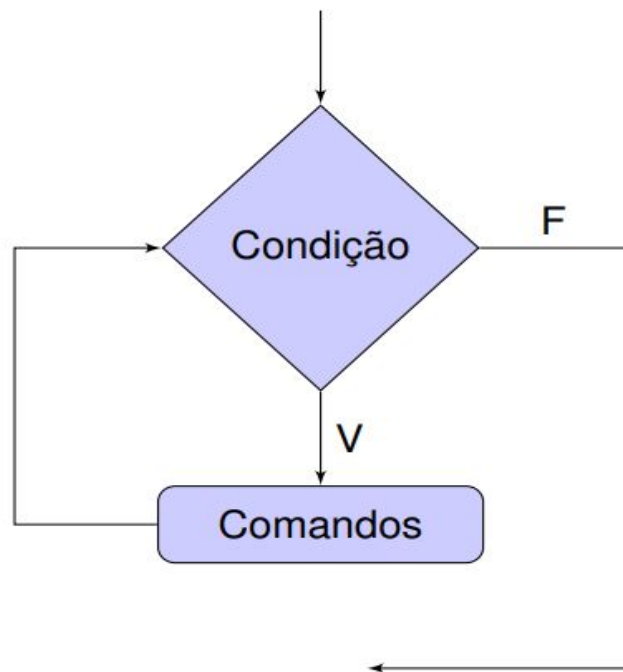
- Em JS é usada quando é preciso realizar uma repetição até que uma dada condição seja verdadeira, senão finaliza a repetição. Neste caso, é feito um teste lógico no início.

while (expressão) {

Comandos a executar enquanto verdadeira a condição;

}

Fluxograma **while**



Estrutura do - While

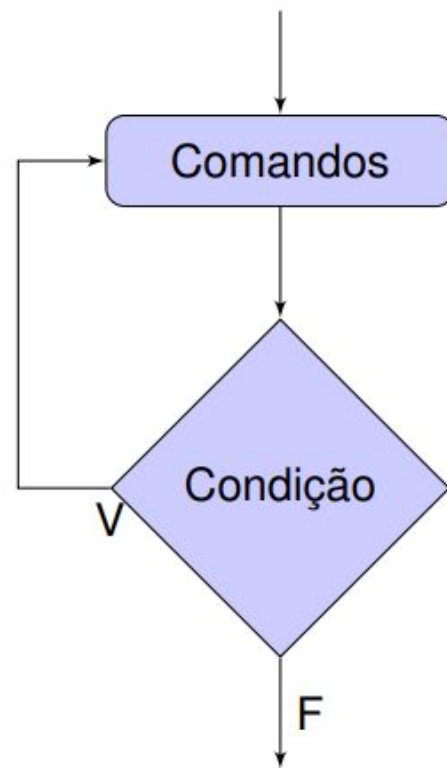
- Em JS é usada quando é preciso realizar **pelo menos uma vez os comandos** e depois é feito o teste de repetição. Neste caso, é feito um teste lógico no final.

do{

Comandos a executar;

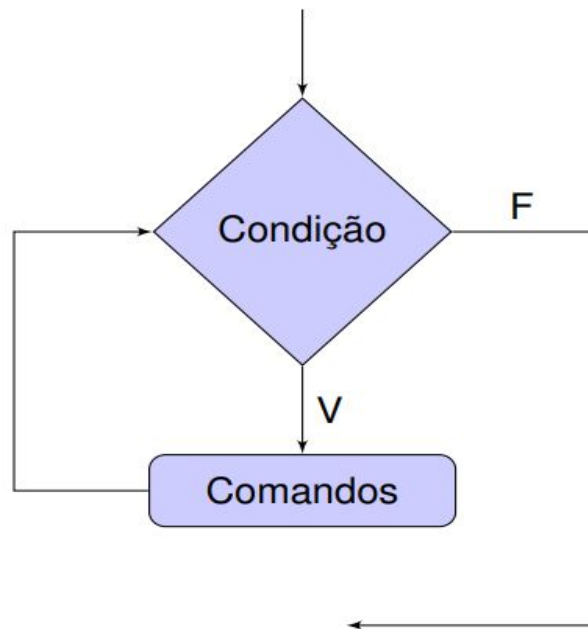
} **while** (expressão)

Fluxograma **do-while**

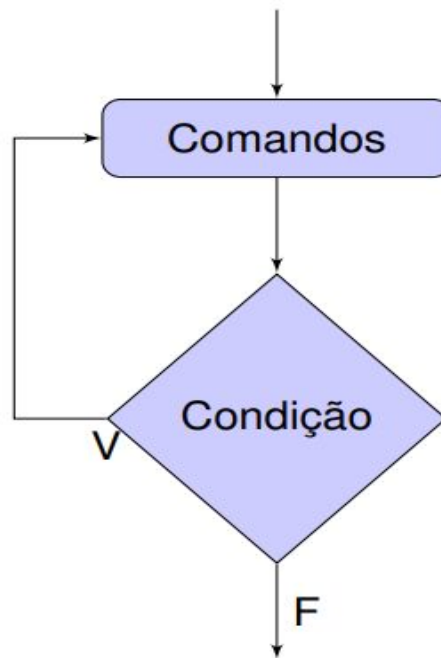


Estrutura While x do - While

Fluxograma **while**



Fluxograma **do-while**



Uso de Comentários em Programação

- O uso de comentários no código, em geral, visa:
- Explicar o algoritmo ou a lógica usada, mostrando o objetivo de uma variável, método, classe, entre outros;
- Documentar o projeto, descrevendo a especificação do código.

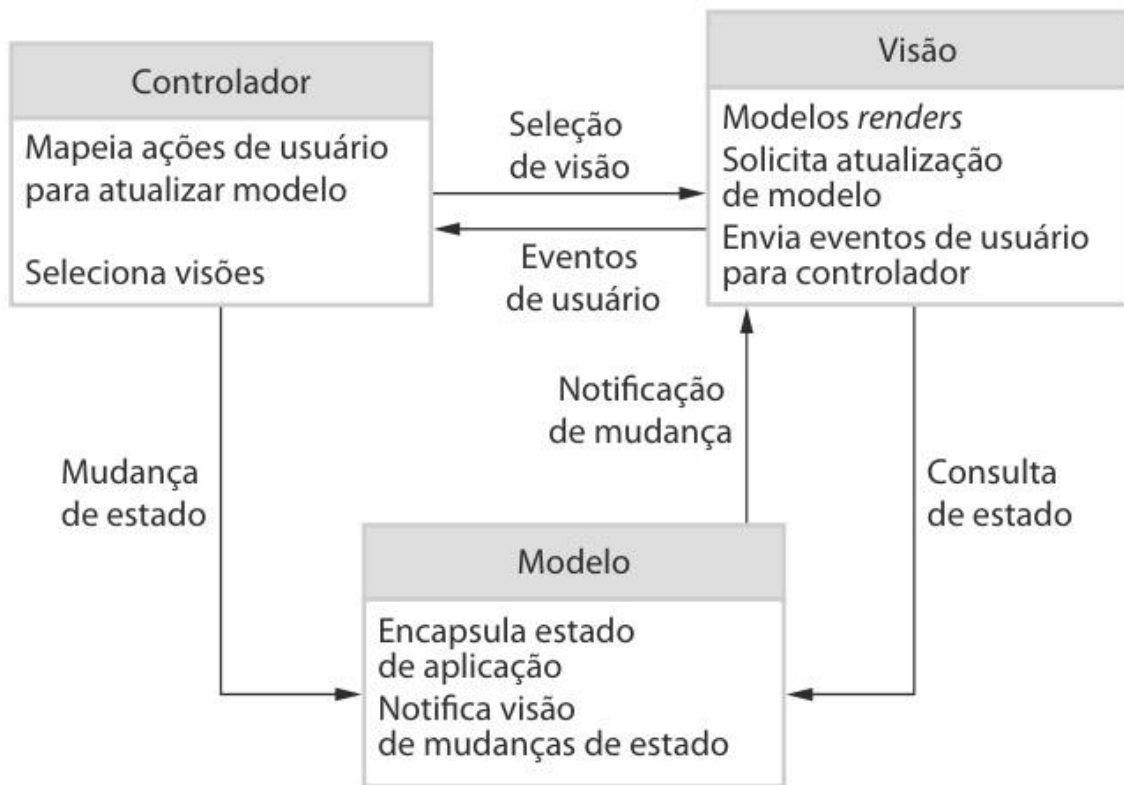
Arquitetura de Software

- A **arquitetura de software** é importante, pois afeta o **desempenho e a robustez**, bem como a capacidade de distribuição e de manutenibilidade de um sistema (Sommerville, 2011).
- As vantagens de comunicar e documentar a arquitetura de software são: Comunicação de **stakeholders**, Análise de sistema, **Reuso** em larga escala.

Arquitetura de Software

- As arquiteturas de sistema são modeladas por meio de diagramas de blocos simples.
- No diagrama, cada caixa representa um componente.
- Caixas dentro de caixas indicam que o componente foi decomposto em sub componentes.
- As setas significam que os dados e/ou sinais de controle são passados de um componente a outro na direção das setas.

Arquitetura de Software - Exemplo



Arquitetura de Software

- O projeto de arquitetura **é um processo criativo** no qual o analista projeta uma organização de sistema para satisfazer aos **requisitos funcionais e não funcionais** de um sistema.
- Por ser um processo criativo, as atividades no âmbito do processo dependem do tipo de sistema a ser desenvolvido, a formação e experiência do arquiteto de sistema e os requisitos específicos para o sistema.

Arquitetura de Software

- A arquitetura de um sistema de software pode se basear em um determinado padrão ou estilo de arquitetura.
- Um **padrão de arquitetura** é uma descrição de uma organização do sistema, como uma organização cliente-servidor ou uma **arquitetura em camadas**.
- Os **padrões de arquitetura** capturam a **essência** de uma arquitetura que tem sido usada em diferentes sistemas de software.

Arquitetura de Software

- Devido à estreita relação entre os **requisitos não funcionais** e a **arquitetura do software**, o estilo e a estrutura da arquitetura particular que o analista escolhe para um sistema devem depender dos **requisitos não funcionais do sistema**, como:
 - **Desempenho;**
 - **Proteção;**
 - **Segurança;**
 - **Disponibilidade;**
 - **Manutenção.**

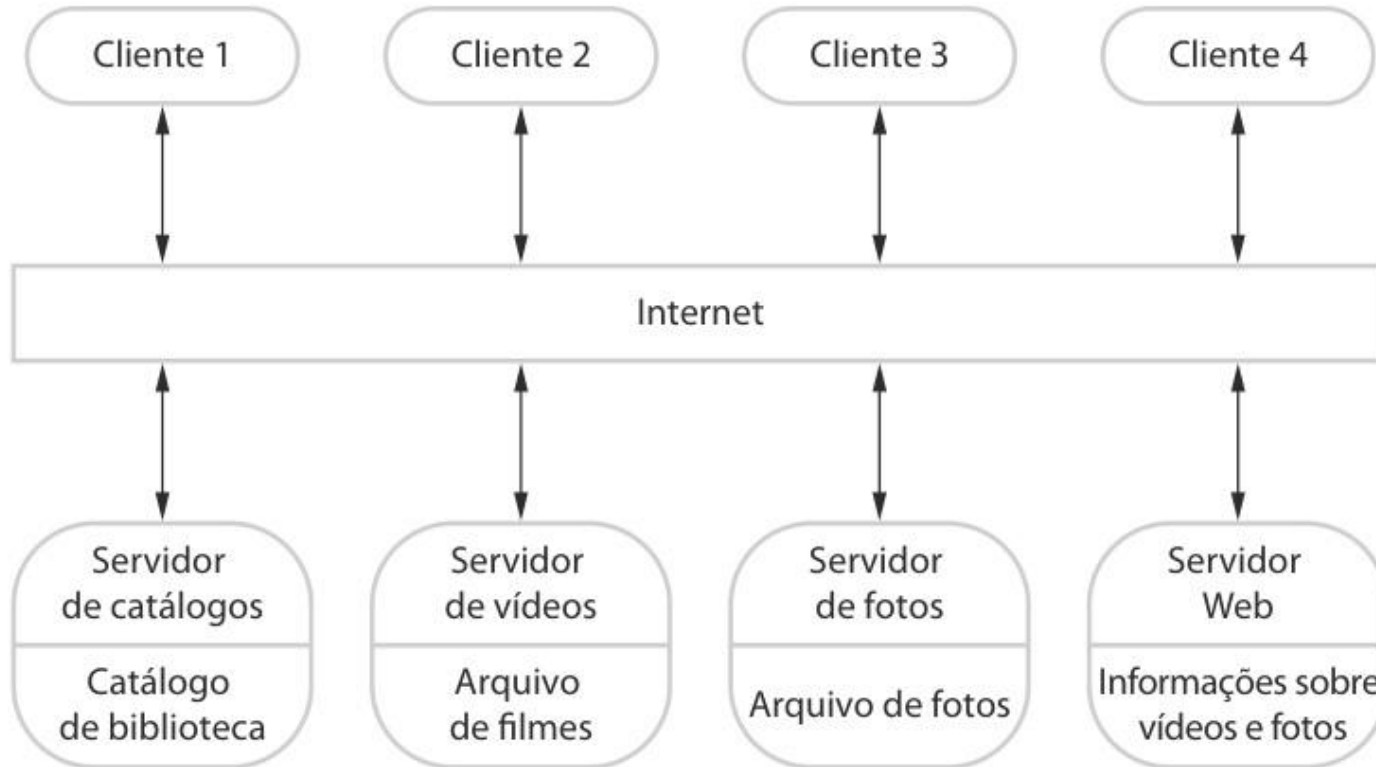
Padrões de Arquitetura de Software

- A ideia de padrões como uma forma de **apresentar, compartilhar e reusar** o conhecimento sobre sistemas de software hoje em dia é amplamente empregada.
- Um padrão de arquitetura pode ser considerado como uma **descrição abstrata, estilizada**, com uso de boas práticas já experimentadas e testadas em diferentes sistemas e ambientes.

Padrões de Arquitetura de Software

- Um **padrão de arquitetura** (*Design Pattern*) deve descrever uma organização de sistema bem-sucedida em sistemas anteriores.
- Deve incluir informações de quando o uso desse padrão é adequado, e seus pontos fortes e fracos.
- Existem alguns padrões de arquitetura de software que são empregados, como: **Arquitetura em Camadas**; Arquitetura de Repositório; Arquitetura Cliente-Servidor; Arquitetura de Filtro e Duto.

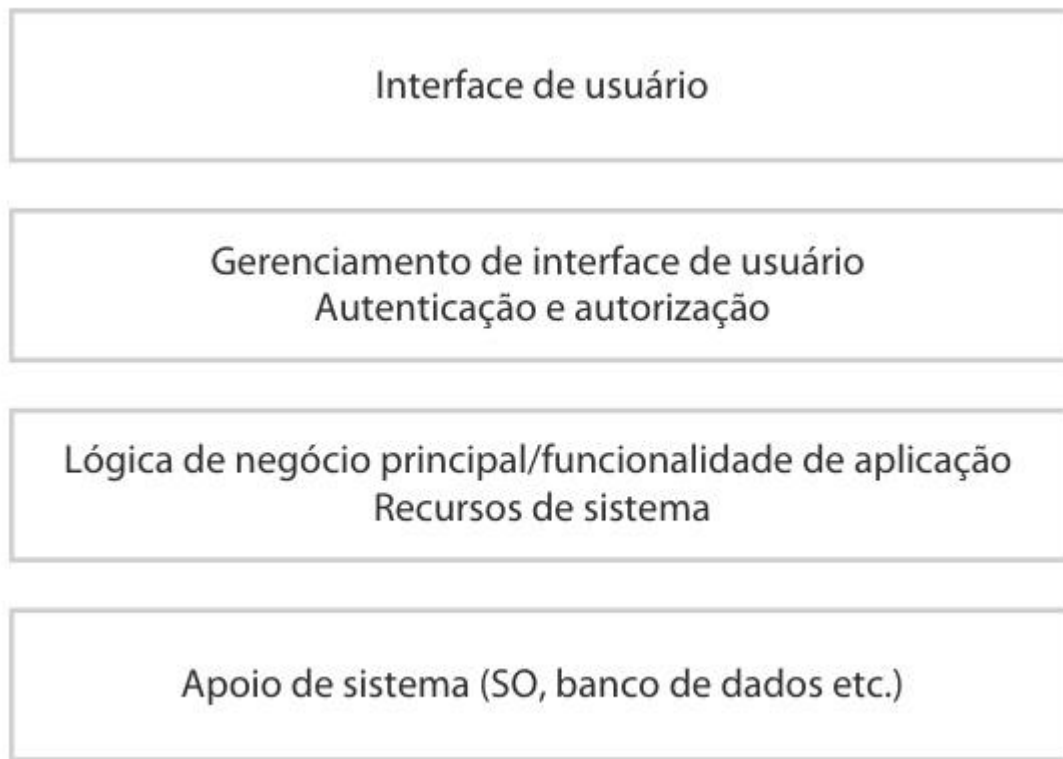
Arquitetura Cliente - Servidor - Exemplo



Arquitetura em Camadas

- Organiza o sistema em **camadas com a funcionalidade** relacionada associada a cada camada.
- Uma camada **fornece serviços** à camada acima dela.
- Os níveis mais baixos de camadas representam os principais serviços suscetíveis de serem usados em todo o sistema.
- É usada na construção de novos recursos em cima de sistemas existentes.

Arquitetura em Camadas

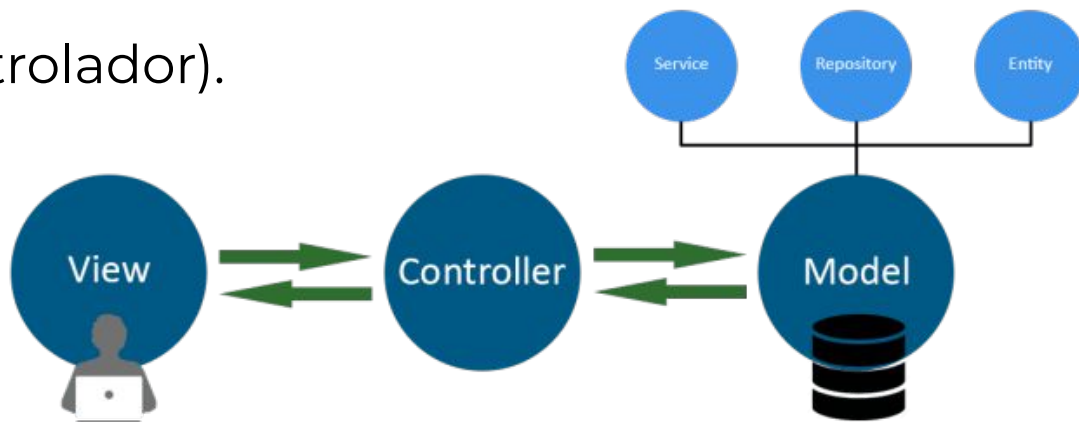


Arquitetura MVC

- É um **padrão de arquitetura** de software (*Design Pattern*), que divide a aplicação em 3 camadas.
- O MVC sugere uma maneira para o analista pensar na divisão de responsabilidades.
- O princípio básico do MVC é a divisão da aplicação em três camadas: a camada de interação do usuário (**View**), a camada de manipulação dos dados (**Model**) e a camada de controle (**Controller**).

Arquitetura MVC

- **MVC** é uma sigla em inglês para:
- **M** – Model (modelo);
- **V** – View (visualização);
- **C** – Controller (controlador).



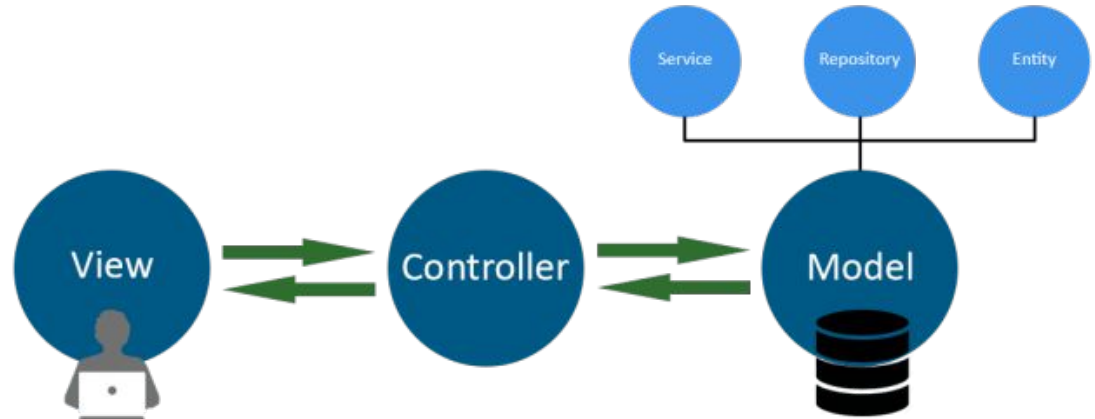
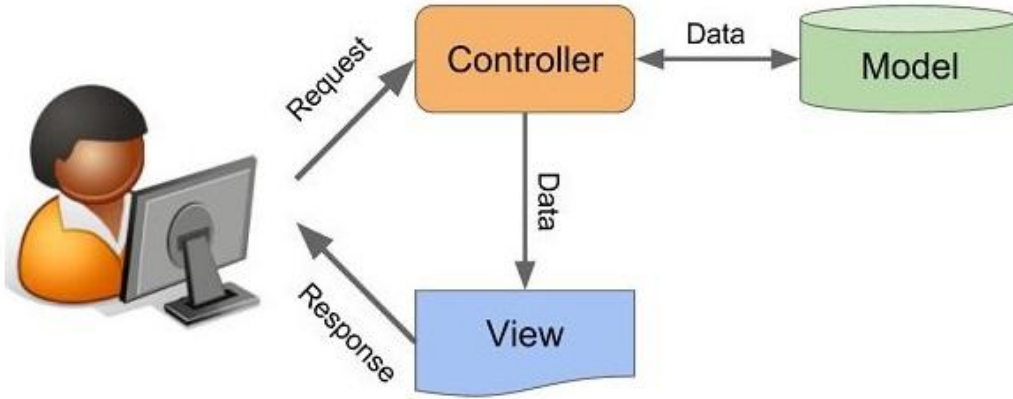
Arquitetura MVC

- A arquitetura MVC foi criada em 1979 pelo cientista da computação norueguês Trygve Reenskaug, que na época trabalhava na Xerox PARC.
- Aliás, a implementação do recurso foi demonstrada no artigo *“Applications Programming in Smalltalk-80: How to use Model-View-Controller”*.
- A proposta do cientista da computação era criar um padrão de arquitetura que separasse o projeto em camadas, reduzindo assim a dependência entre elas.

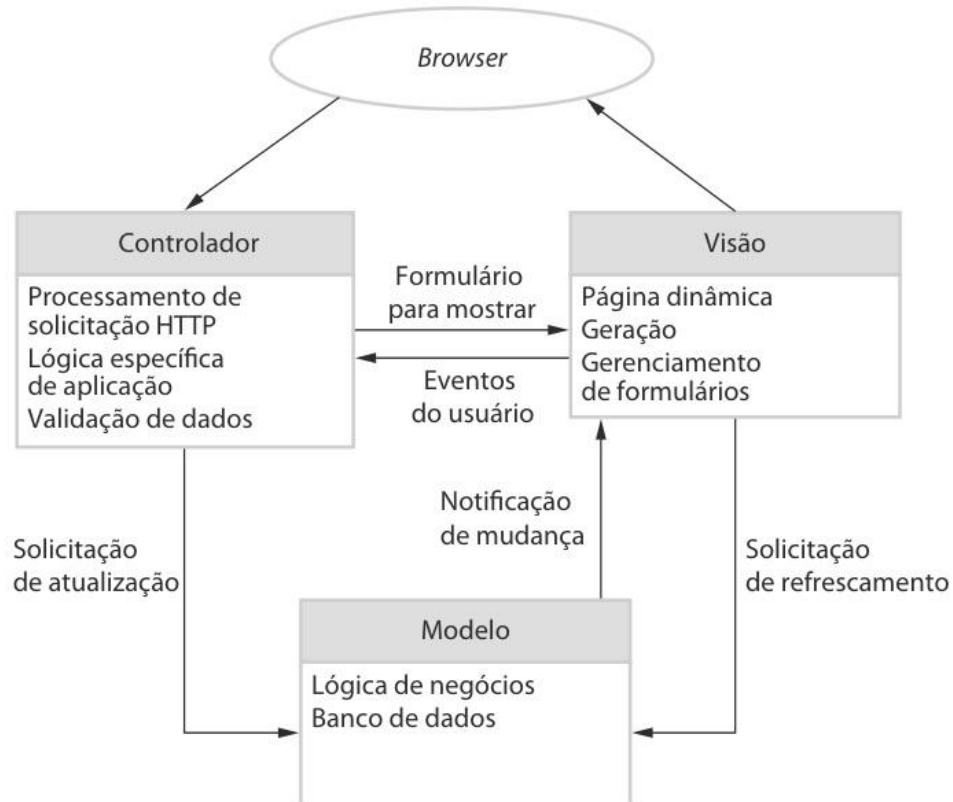
Arquitetura MVC - Responsabilidades

- Quando se fala sobre o MVC, cada uma das camadas apresenta as seguintes responsabilidades:
- **Model**: é responsável por **representar o negócio**, assim como o acesso e manipulação dos dados na sua aplicação;
- **View**: é responsável pela **interface do sistema**, mostrando as informações do Model para o usuário;
- **Controller**: é responsável por **ligar** a Model e a View, fazendo com que os Models possam ser repassados para as Views e vice-versa.

Arquitectura MVC - Responsabilidades



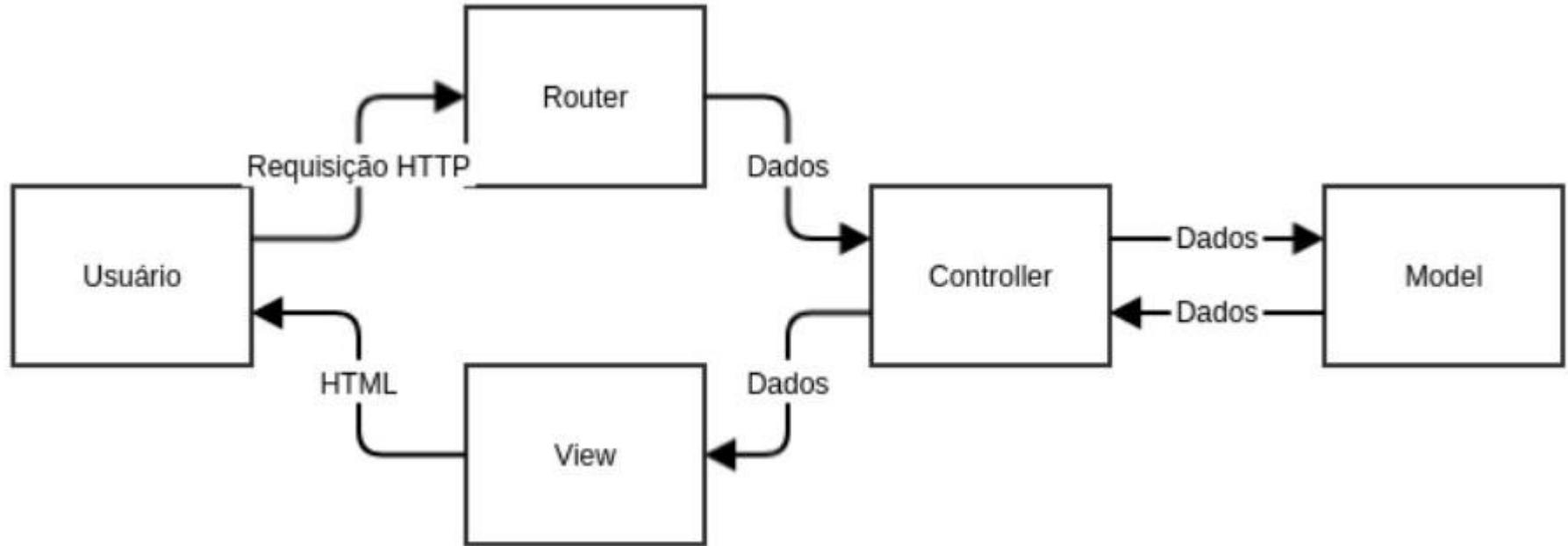
Arquitetura MVC - Responsabilidades



Vantagens da Arquitetura MVC

- A arquitetura MVC proporciona mais agilidade ao trabalho do desenvolvedor, além das seguintes vantagens:
- Agilidade na atualização da interface da aplicação;
- Facilidade de manutenção do código;
- Facilidade na implementação de camadas de segurança;
- Integração de equipes de desenvolvedores.

Arquitetura MVC - Node JS



Node.JS

- **Node.js®** é um **runtime** JavaScript desenvolvido com o **Chrome V8 JavaScript engine**.
- O Node.JS é um **interpretador Javascript** que não depende do navegador, ou seja, é desvinculado do navegador.



Node.JS

- Como um ambiente de **execução JavaScript assíncrono**, no lado do servidor, **orientado a eventos**, que é projetado para desenvolvimento de aplicações escaláveis de rede.
- O Node.js **não** é uma linguagem, **não** é um framework Javascript.
- O Node.js **é uma ferramenta** de código aberto, que permite **rodar o Javascript** fora do navegador (browser).

Node.JS

- O javascript era apenas **“lido”** e **“interpretado”** por navegadores da web.
- Neste caso, apenas os navegadores, como o Chrome, Firefox e outros, eram capazes de executar o Javascript e realizar alguma ação.

Node.JS

- Assim o Javascript era considerado apenas uma linguagem do lado do cliente, ou seja, **client-side**, que rodava apenas no navegador do usuário, sendo usado para manipular uma página de um site e realizar alguns eventos.



Node.JS

- Com a separação do motor de Javascript chamado V8, para leitura e interpretação do código fora do navegador, tem-se o javascript no lado do servidor, ou seja, **server-side**.



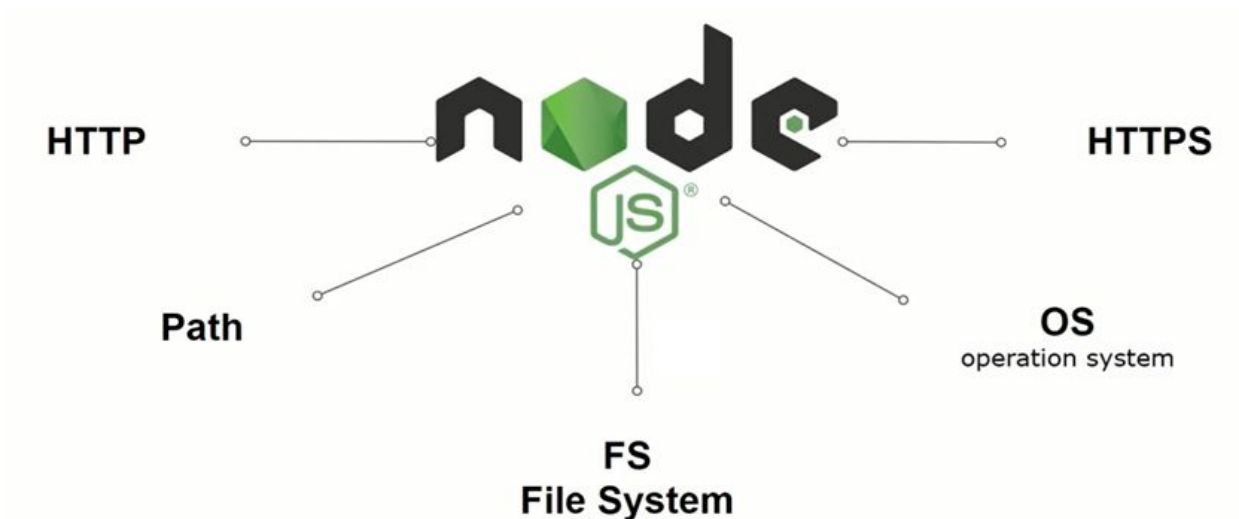
Node.JS

- Neste caso, agora o Node realiza operações como por exemplo: criar, abrir, ler e escrever arquivos.
- Coletar dados e manipular banco de dados.
- Ainda, é possível executar o Node em diferentes **plataformas**.



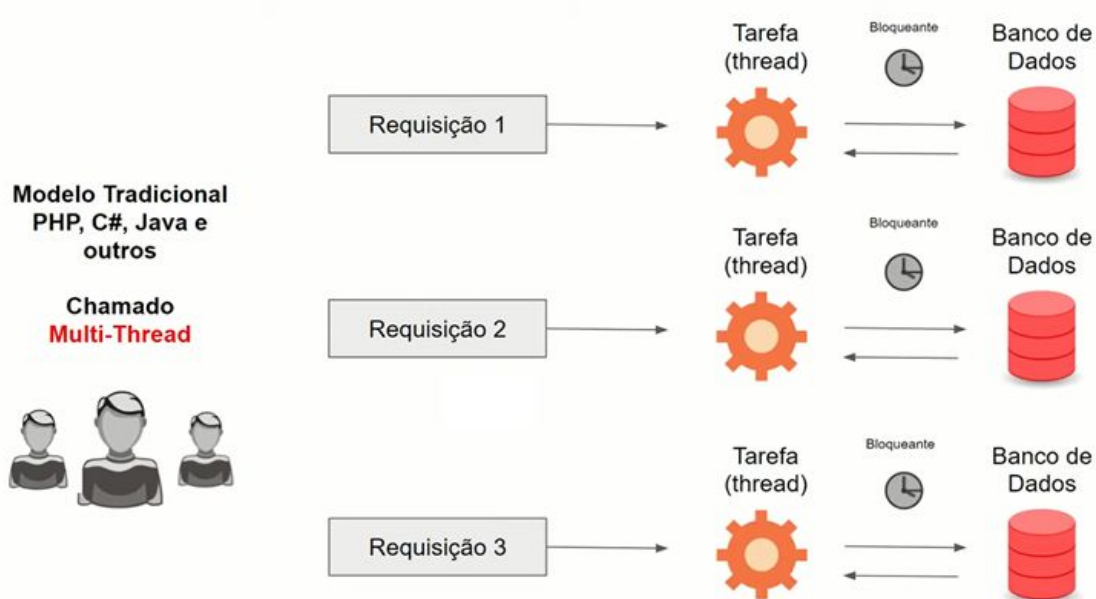
Node.JS

- O Node é formado por módulos, que possibilitam interagir com **requisições HTTP**, com o sistema operacional e arquivos do sistema, além de manipulação de dados com Banco de Dados.



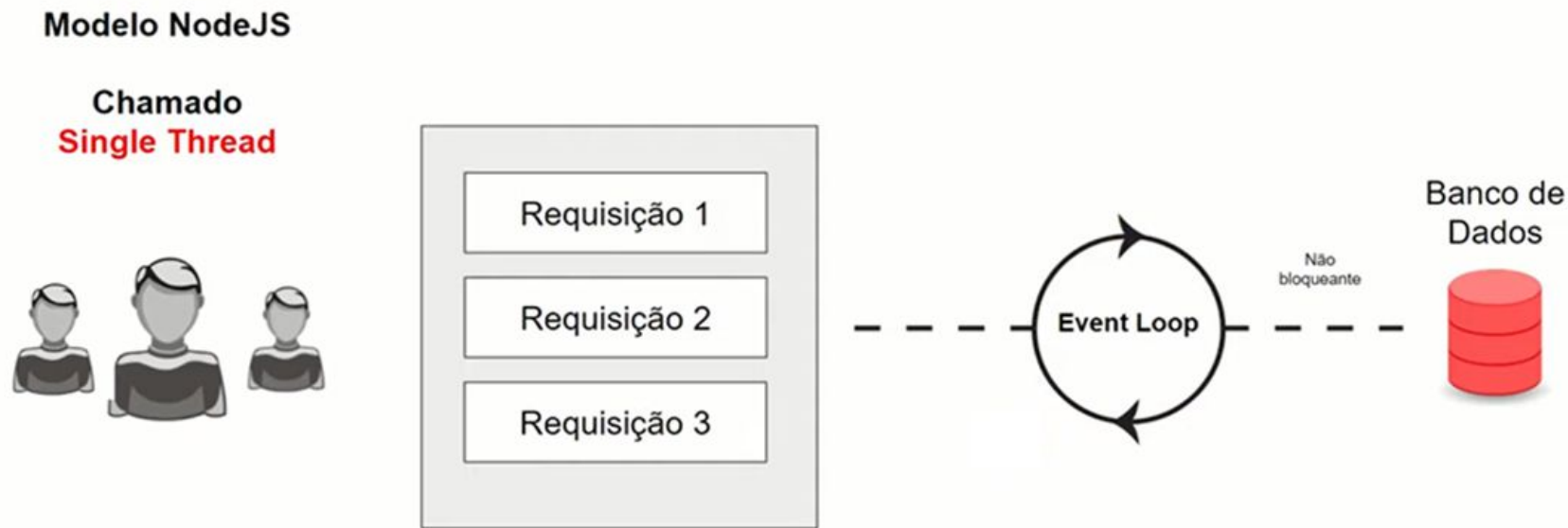
Node.JS

- Em um sistema de acesso tradicional com banco de dados, são geradas tarefas, a partir de uma requisição para manipulação de dados.



Node.JS

- Com o Node, esta manipulação é feita através de requisições que entram em uma fila de execução.



Node.JS

- Para execução do Node, através do comando no terminal, solicita-se que seja executado um arquivo Javascript.



Node.JS

- Exemplo de comandos Javascript para execução com o Node.
- `console.log("Olá Mundo !")`
- `console.log("\n")`

- `let nome = 'Carlos'`
- `console.log("Olá: " + nome + " " + " Tudo bem contigo?")`
- `console.log("\n ")`

Node.JS

- Usando o conceito de uma função para fazer uma operação aritmética.

```
function soma(x,y){ return x+y }
```

```
let x = 5, y = 3
```

```
let resposta = soma(x,y)
```

```
console.log("O resultado da soma de "+x+" com "+y+" é = "+resposta)
```

Node.JS

- O **NPM** é o Gerenciador de Pacotes do Node (**N**ode **P**ackage **M**anager).
- Ele permite instalar, desinstalar e atualizar dependências em uma aplicação por meio de uma instrução na linha de comando.
- Sempre que um projeto é criado por meio do gerenciador, é adicionado um arquivo chamado **package.json**, que contém a relação dos pacotes instalados no ambiente.

Node.JS

- Considerando o desenvolvimento de um servidor web, o Node possui vários benefícios:
- Alta performance, pois otimiza a taxa de transferência de dados e a escalabilidade em aplicações web.
- O código é escrito em “JavaScript”, o que permite usar a mesma linguagem no lado do cliente e no lado do servidor.

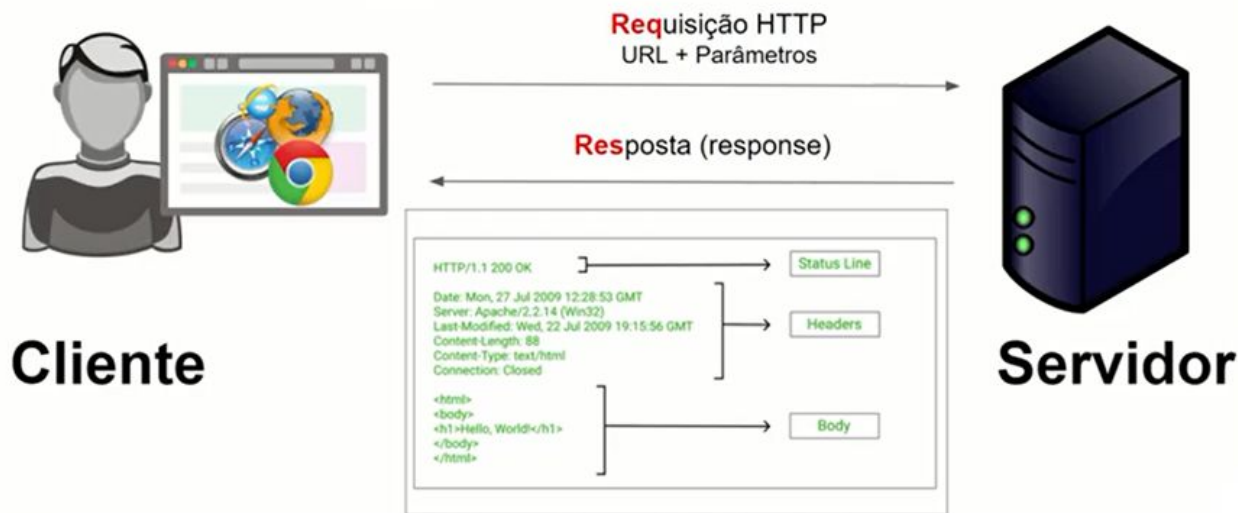
Node.JS

- O Gerenciador de Pacotes do Node (NPM, na sigla em inglês) provê acesso à muitos pacotes que podem ser reutilizados, possibilitando também a gestão de dependências.
- É portátil, com versões para diferentes sistemas operacionais (diferentes plataformas), como Microsoft Windows, OS X, Linux, e outros.

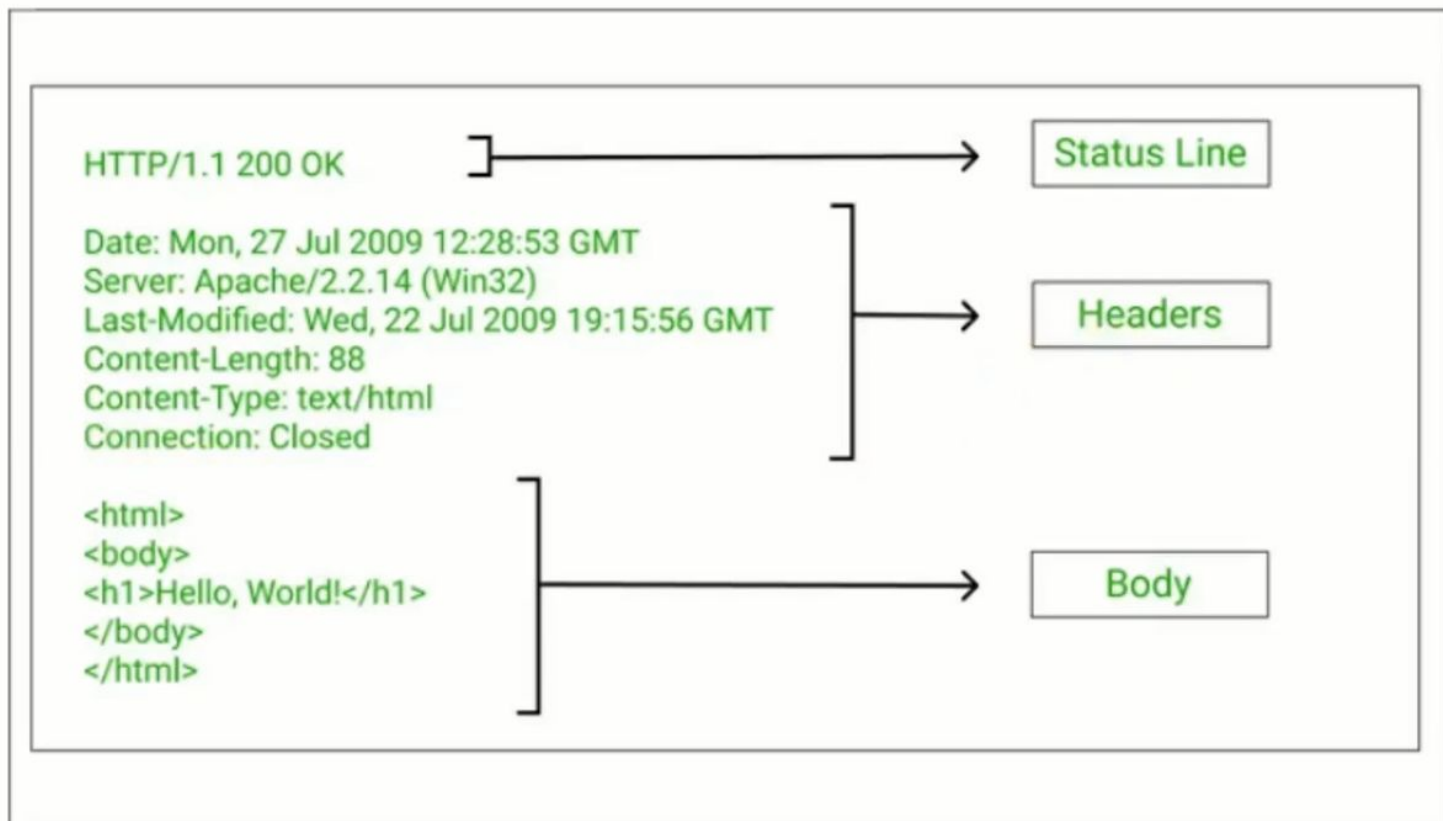
Node.JS

- Quando se trata da comunicação em um sistema WEB, o Node aplica o protocolo HTTP ou HTTPS para troca de mensagens.

HTTP — HyperText Transfer Protocol



Node.JS



Node.JS

- Exemplo:
- `const http = require("http");`
- `const PORT = 3000`
- `const hostname = '127.0.0.1'`
- `http.createServer((req, res)=> {`
- `res.writeHead(200, {'Content-Type': 'text/plain'});`
- `res.end('Hello World!\n');`
- `}).listen(PORT, hostname);`
- `console.log(`Servidor executando em ${hostname}:${PORT}`);`

Node.JS

- Exemplo:
- `const http = require("http");`
- `const PORT = 3000`
- `const hostname = '127.0.0.1'`
- `http.createServer((req, res)=> {`
- `res.writeHead(200, {'Content-type':'text/html; charset=utf-8'})`
- `res.end('<h1>Olá Mundo!</h1> <br> <h2>Desenvolvimento WEB</h2>'))).listen(PORT, hostname);`
- `console.log(`Servidor executando em ${hostname}:${PORT}`);`

Web Framework - Express

- Algumas tarefas no desenvolvimento web possuem mais facilidades quando se utilizam **framework**, ao invés de usar diretamente o Node.
- Um exemplo é o Framework **Express**.
- Ele é considerado um Framework rápido e muito utilizado em conjunto com o Node.js, facilitando o desenvolvimento de aplicações back-end.

Express JS



Web Framework - Express

- O **Express** foi criado em 2010 por TJ Holowaychuk, sendo que foi lançado como software livre e de código aberto sob a licença MIT.
- O **Express** é escrito em JavaScript, sendo que é utilizado por diversas empresas como a Fox Sports, PayPal, IBM, Uber, entre outras.
- O **Express** é, portanto, um **framework para aplicativo da web do Node.js** que fornece um conjunto de recursos para desenvolvimento web.

Web Framework - Express

- Algumas **características** do Express são:
- Possui um sistema de rotas;
- Possibilita o tratamento de exceções dentro de uma aplicação;
- Permite a integração de vários sistemas de “templates” que facilitam a criação de páginas web para suas aplicações;
- Gerenciamento de diferentes requisições HTTP, como: GET e POST, por exemplo;
- Possibilita a criação rápida de aplicações com o uso de um conjunto pequeno de arquivos e pastas.

Web Framework - Express

- Em um sistema web convencional, um aplicativo da Web aguarda pedidos HTTP do web browser, ou seja, do cliente
- Quando um pedido é recebido pelo servidor, o aplicativo descreve quais ações são necessárias com base no padrão de URL e informações associadas contidas em dados do tipo **GET** ou **POST**.
- Dependendo do que é necessário, pode-se ler ou escrever informações em um banco de dados ou executar outras tarefas necessárias para satisfazer a solicitação.

Web Framework - Express

- O Express **fornece métodos** para especificar qual função é chamada quando chega requisição HTTP, como **GET e POST**, além de rotas e métodos para especificar o **mecanismo de modelo ("view")** usado, onde o modelo arquivos estão localizados e qual modelo usar para renderizar uma resposta.
- Para utilizar o Express é preciso instalar o módulo.
- **npm install --save express**

Setup e configuração - express()

- O **require** é uma forma que foi desenvolvida para Node.js de **importar e exportar módulos** em uma aplicação.
- Exemplo:
- `const express = require('express')` => importa módulo express
- `const app = express()` => cria uma aplicação express

Referência:

https://developer.mozilla.org/pt-BR/docs/Learn/Server-side/Express_Nodejs/Introduction

Setup e configuração - express()

- `const conn = require('./db/conn')`
- Importa o módulo de comunicação com o banco de dados, módulo do programa [implementado pelo programador](#).

Referência:

sem referência externa;

Setup e configuração - express()

- `const Usuario= require('./models/Usuario')`
- Importa o módulo de models - Modelo para troca de informação com o banco de dados em um Modelo de Objeto Relacional - ORM (Object Relational Mapper), sendo este modelo [implementado pelo programador](#).

Referência:

sem referência externa;

Setup e configuração - express()

- const **PORT = 3000**
- const **hostname = 'localhost'**
- A aplicação NODE fica ouvindo (listen) um endereço (localhost) e uma porta especificada (PORT = 3000) e retorna uma mensagem HTTP com conteúdo (log) **implementado pelo programador**.

Referência:

<https://nodejs.org/en/docs/guides/getting-started-guide>

Setup e configuração - express()

- `app.use(express.urlencoded({extended:true}))`
- Esta é uma função de **middleware** integrada no Express. Ele analisa as solicitações recebidas com os dados preenchidos no corpo da mensagem em uma codificação (UTF-8 por exemplo).

Referência:

<https://expressjs.com/en/4x/api.html>

Setup e configuração - express()

- `app.use(express.urlencoded({extended:true}))`
- **extended: true** => A sintaxe “estendida” permite que objetos e arrays sejam codificados em dado formato URL, semelhante ao JSON.

Referência:

<https://expressjs.com/en/4x/api.html>

Setup e configuração - express()

- `app.use(express.json())`
- Esta é uma função de **middleware** integrada no Express. Ele analisa as solicitações recebidas com dados JSON e é baseado no corpo da mensagem.

Referência:

<https://expressjs.com/en/4x/api.html>

Setup e configuração - express()

- `app.use(express.static('public'))`
- Esta é uma função de **middleware** integrada no Express. O argumento raiz especifica a pasta raiz a partir do qual serão “servidos” os recursos estáticos.

Referência:

<https://expressjs.com/en/4x/api.html>

Setup e configuração - sequelize()

- **sequelize.authenticate().then().catch()**
- Esta é uma função do **sequelize** de teste de conexão com o banco de dados, verificando se o banco de dados existe. Se verdadeiro **then()**, mas se tem erro **catch()**.

Referência:

<https://sequelize.org/docs/v6/getting-started/#testing-the-connection>

Setup e configuração - express()

- `app.listen(PORT, hostname, ()=>`
- `console.log(`Servidor rodando em ${hostname}:${PORT}`)`
- `})`
- Esta é uma função de **middleware** integrada no Express.
Vincula e escuta conexões no “host” e “port” especificados.

Referência: <https://expressjs.com/en/4x/api.html>

Setup e configuração - sequelize()

- `const { Sequelize } = require('sequelize')`
- Para se conectar ao banco de dados, criando uma instância do Sequelize. Pode ser feito passando os parâmetros de conexão separados para o construtor Sequelize ou passando um único URI de conexão.

Referência:

<https://sequelize.org/docs/v6/getting-started/#connecting-to-a-database>

Setup e configuração - sequelize()

- **sequelize.authenticate().then().catch()**
- Esta é uma função do **sequelize** de teste de conexão com o banco de dados, verificando se o banco de dados existe. Se verdadeiro **then()**, mas se tem erro **catch()**.

Referência:

<https://sequelize.org/docs/v6/getting-started/#testing-the-connection>

Setup e configuração - sequelize()

- **conn.sync().then().catch()**
- Esta é uma função do **sequelize** de sincronização do modelo com o banco de dados, verificando se a tabela do banco existe, se tem as mesmas colunas e garante que não haja diferenças. Se verdadeiro **then()**, mas se tem erro **catch()**.

Referência:

<https://sequelize.org/docs/v6/core-concepts/model-basics/#model-synchronization>

Setup e configuração - sequelize()

- `const db= new Sequelize('banco_dados','usuario','senha', {`
- `host: 'localhost',`
- `dialect: 'postgres',`
- `port: 5432 })`
- Parâmetros de conexão sendo passados separados para o construtor Sequelize em uma conexão.

Referência:

<https://sequelize.org/docs/v6/getting-started/#connecting-to-a-database>

Setup e configuração - sequelize()

- `const { DataTypes } = require('sequelize')`
- `const db = require('../db/conn')`
- Para se conectar ao banco de dados, com parâmetros de tipos de dados e com a conexão implementada pelo programador.

Referência:

<https://sequelize.org/docs/v6/getting-started/#connecting-to-a-database>

Setup e configuração - sequelize()

- `const Usuario = db.define('usuarios',{`
- `nome: {`
- `type: DataTypes.STRING,`
- `},`
- `idade: {`
- `type: DataTypes.INTEGER,`
- `},`
- `{`
- `timestamps: false,`
- `tableName: 'nome_tabela'`
- `})`

Setup e configuração - sequelize()

- **Model Definition** (definição de modelo):
- Os modelos podem ser definidos de duas maneiras;
- Será utilizado o modelo “**sequelize.define**”, através do objeto db que contém a conexão do banco.
- exemplo:
- **const Usuario = db.define()**.

JSON - JavaScript Object Notation

- O **JSON** (**J**ava**S**cript **O**bject **N**otation) é um formato baseado em texto padrão para representar dados estruturados com base na sintaxe do objeto JavaScript.
- É comumente usado para transmitir dados em aplicativos da Web.
- O arquivo contém apenas texto e usa a extensão “.json”.
- <https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/Objects/JSON>

JSON - JavaScript Object Notation

- O **JSON** é um formato de dados baseado em texto seguindo a sintaxe do objeto JavaScript, que foi popularizada por Douglas Crockford.
- Mesmo que se assemelhe à sintaxe literal do objeto JavaScript, ele pode ser usado de modo independente, e muitos ambientes de programação possuem a capacidade de ler e gerar JSON.

JSON - JavaScript Object Notation

- O **JSON** existe como *uma string* — útil quando se deseja transmitir dados por uma rede.
- O **JSON** precisa ser convertido em um objeto JavaScript nativo quando se deseja acessar os dados.
- Um objeto **JSON** pode ser armazenado em seu próprio arquivo, que é basicamente apenas um arquivo de texto com uma extensão de *“.json”*.

Sintaxe do JSON

- Existem 2 elementos em um objeto JSON: **Chave/Valor**.
- **Chaves**: devem ser strings separadas aspas;
- **Valores**: são um tipo válido de dados JSON no formato de:
 - *array, object, string, boolean, number ou null*.
- Um objeto JSON inicia e termina com **chaves** “{ }” e tem no arquivo **2 ou mais** pares de **chave/valor** separados por **vírgula**, onde cada chave é **seguida por dois pontos** para diferenciar o valor.

Sintaxe do JSON

- Exemplo:

```
{ "nome":"Palio",  
  "cor":"Azul",  
  "motor":1.6,  
  "opcionais": {  
    "roda":"liga leve",  
    "aro":15,  
    "air_bag":"frente total"  }  
}
```

```
{  
  "nome":"Palio",  
  "cor":"Azul",  
  "motor":1.6,  
  "opcionais":{  
    "roda":"liga leve",  
    "aro":15,  
    "air_bag":"frente total"  
  }  
}
```

Comparação JSON x Objeto

```
{  
  "nome": "Palio",  
  "cor": "Azul",  
  "motor": 1.6,  
  "opcionais": {  
    "roda": "liga leve",  
    "aro": 15,  
    "air_bag": "frente total"  
  }  
}
```

```
const Carro = {  
  nome: "Palio",  
  cor: "Azul",  
  motor: 1.6,  
  opcionais: {  
    roda: "liga leve",  
    aro: 15,  
    air_bag: "frente total"  
  }  
}
```

Manipulando dados JSON

- Para converter um **objeto** em um **string JSON**, utiliza-se o comando “JSON.stringify(**nome_objeto**)” em Javascript.
- Para converter uma **string** para o formato **objeto JSON**, utiliza-se o comando “JSON.parse(**str_json**)” em Javascript.
- A figura a seguir mostra um código contendo um objeto que é convertido em uma string no formato JSON e depois é convertida em um dado do tipo JSON.

Exemplo de conversão JSON

```
const Carro = {
  nome: "Palio",
  cor: "Azul",
  motor: 1.6,
  opcionais: {
    roda: "liga leve",
    aro: 15,
    air_bag: "frente total"
  }
}

const str_json = JSON.stringify(Carro)

console.log(Carro)
console.log('-----')
console.log(str_json)
console.log('-----')

const obj_json = JSON.parse(str_json)
console.log(obj_json)
console.log(obj_json.nome)
```


Utilização do JSON

- O JSON é muito utilizado em várias aplicações. Seus principais usos incluem:
- **Comunicação de dados na web:** para transmitir dados entre um servidor e um cliente em aplicações web, com o uso do http para APIs, onde os dados são usados no formato JSON;
- **Armazenamento de Configurações:** usado para guardar configurações e preferências do usuário (fácil manutenção).

Utilização do JSON

- **Armazenamento de dados em bancos NoSQL:** os bancos de dados MongoDB e CouchDB, armazenam nativamente os dados em formato JSON;
- **Troca de dados entre sistemas:** é muito usado para troca de dados entre diferentes sistemas e serviços (integração).
- **Configuração de APIs:** usam JSON para definir a estrutura dos dados que esperam receber em solicitações e as respostas que retornam (integração).

Fetch API

- A **Fetch API** é uma interface JavaScript moderna que fornece uma maneira fácil e poderosa de fazer solicitações de rede HTTP, como buscar recursos em um servidor web.
- A **Fetch API** opera em promessas (Promise).
- A **Fetch API** é [nativamente](#) suportada em [navegadores](#) modernos e também pode ser usada em ambientes Node.js com o auxílio de bibliotecas como “**node-fetch**”.

Fetch API

```
fetch('http://api.example.com/data')  
  .then(response => response.json() ) // converter para json  
  .then(data => console.log(data))    //imprimir dados no console  
  .catch(error => console.error('Erro de solicitação:, error)) // tratar erros
```

fetch(): é o método principal usado para fazer a solicitação de rede.

response: representa a resposta a uma solicitação Fetch e possui os métodos para acessar o corpo da resposta em formatos (JSON, texto, ...)

then(): é usado para encadear operações assíncronas em uma promise, permitindo manipular o resultado da solicitação e o erro **catch()**.

Exemplo de aplicação JSON

- Obtendo o endereço a partir do CEP, em uma base de dados na web que disponibiliza as informações no formato JSON.

JSON

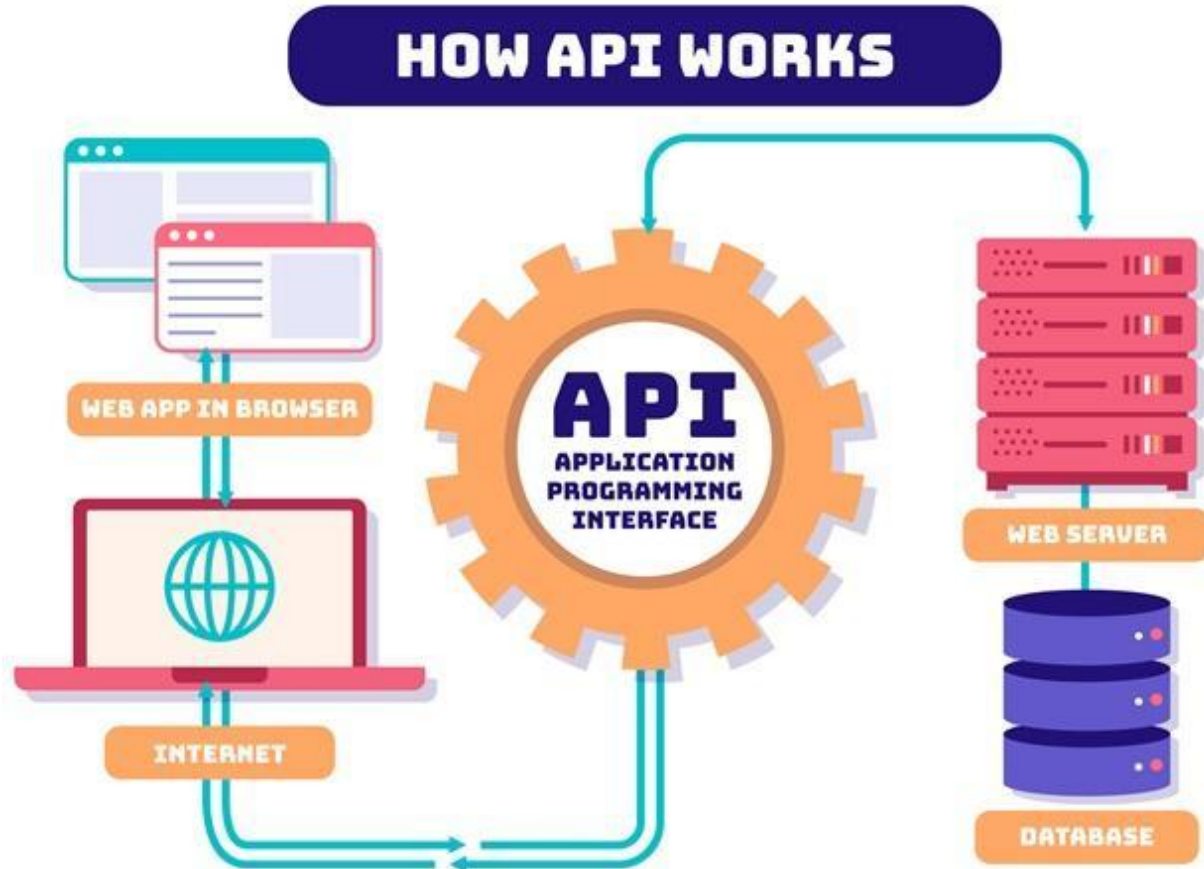
Obtendo um CEP site **ViaCEP** <https://viacep.com.br/>

Digite um cep:

Pesquisa

Rua Max Colin
América
Joinville
SC
89204-041

API - Application Programming Interface



API - **A**pplication **P**rogramming **I**nterface

- API ou Interface de Programação de Aplicações, é um conjunto de protocolos, rotinas e ferramentas que permitem que diferentes aplicativos de software se comuniquem entre si.
- As APIs agem como **intermediárias** entre aplicativos de software, permitindo que eles troquem dados e acessem as funcionalidades uns dos outros.

API - **A**pplication **P**rogramming **I**nterface

- Uma API define um **conjunto específico de regras e padrões** para a construção de aplicativos de software, especificando como os componentes de software devem interagir uns com os outros.
- Ela fornece **uma maneira padronizada para diferentes aplicativos acessarem e compartilharem dados e serviços**, sem exigir que os desenvolvedores desses aplicativos entendam os detalhes de implementação subjacentes.

API REST - **RE**presentational **ST**ate **T**ransfer

- API **REST** é um estilo arquitetural para o desenvolvimento de APIs que segue um conjunto de princípios e restrições.
- Elas são baseadas no protocolo HTTP e são projetadas para **facilitar a comunicação entre sistemas distribuídos na web.**
- Algumas características chave de uma API REST são:
- Baseada em Recursos, Estado Representacional e Operações HTTP.

API REST - **RE**presentational **St**ate **T**ransfer

- **Baseada em Recursos:** Os recursos (entidades) são os elementos principais de uma API REST e representam os objetos ou dados que estão sendo manipulados. Cada recurso é identificado por um URI (Uniform Resource Identifier).
- **Operações HTTP:** As operações **CRUD** são mapeadas para os métodos HTTP: **POST, GET, PUT/PATCH e DELETE**.
- Isso permite que as ações sobre os recursos sejam realizadas de acordo com os padrões do protocolo HTTP.

API REST - **RE**presentational **S**tate **T**ransfer

- **Estado Representacional**: Os recursos são representados de forma independente de estado, o que significa que cada solicitação deve conter todas as informações necessárias para entender e processar a solicitação.
- Isso permite que os sistemas sejam altamente escaláveis e independentes de estado.

API RESTful

- Uma API é pode ser considerada **RESTful** quando ela adere aos princípios e restrições do estilo arquitetural REST e algumas práticas comuns como:
- uso correto dos **verbos HTTP**, nomes de recursos descritivos e semânticos, representação dos recursos em formatos padronizados (JSON ou XML), tratamento de erros e status code, adotar medidas de segurança, como autorização e autenticação.

Operações CRUD e os Verbos HTTP

- **POST**: Utilizado para criar um novo recurso no servidor.
- Os dados para criar o recurso são enviados no corpo da solicitação.

POST /usuários	Cria um novo usuário com os dados fornecidos no corpo da solicitação
-----------------------	--

Operações CRUD e os Verbos HTTP

- **GET**: Utilizado para buscar dados do servidor.
- Em geral, o endpoint deve retornar uma lista de recursos ou um recurso específico, dependendo da URL fornecida.
- Se houver parâmetros de consulta, eles podem ser usados para filtrar os resultados.

GET /usuarios	Retorna todos os usuários
GET /usuario/1	Retorna o usuário com ID 1
GET /usuarios?idade=30	Retorna usuários com idade igual a 30

Operações CRUD e os Verbos HTTP

- **PUT**: Utilizado para atualizar um recurso existente no servidor.
- O cliente envia os dados completos do recurso atualizado no corpo da solicitação.

PUT/usuario/1	Atualiza o usuário com ID 1 com os dados fornecidos no corpo da solicitação
----------------------	---

Operações CRUD e os Verbos HTTP

- **PATCH**: Utilizado para aplicar modificações parciais a um recurso existente no servidor.
- O cliente envia apenas os campos que devem ser modificados.

PATCH /usuario/1	Aplica modificações parciais ao usuário com ID 1, com os campos fornecidos no corpo da solicitação
-------------------------	--

Operações CRUD e os Verbos HTTP

- **DELETE**: Utilizado para excluir um recurso existente no servidor.

DELETE /usuarios/1	Exclui o usuário com ID 1
---------------------------	---------------------------

API RESTFul - exemplo - rota /usuario

- **GET** - Recuperar Todos os Usuários: Front end (**Fetch**):

```
fetch('http://localhost:3000/usuario')
```

```
.then(response => response.json())
```

```
.then(data => console.log(data))
```

```
.catch(error => console.error('Erro ao recuperar usuários:', error))
```

API RESTFul - exemplo - rota /usuario

- **GET** - Recuperar Todos os Usuários: Back end (**express**):

```
app.get('/usuario', async (req, res) => {  
  try {  
    const usuarios = await Usuario.findAll()  
    res.json(usuarios)  
  } catch (error) {  
    console.error('Erro ao recuperar usuários:', error)  
    res.status(500).json({ error: 'Erro ao recuperar usuários' })  
  }  
})
```

API RESTFul - exemplo - rota /usuario

- **GET** - Recuperar dados do Usuário: Front end (**Fetch**):

```
fetch(`http://localhost:3000/usuario?nome=${nomeUsuario}&idade=${idadeUsuario}`)  
  
  .then(response => response.json())  
  
  .then(usuario => {  
  
    const { nome, idade } = usuario;  
  
    console.log('Nome:', nome);  
  
    console.log('Idade:', idade);  
  
  }).catch(error => console.error('Erro ao recuperar nome e idade do usuário:', error));
```

API RESTFul - exemplo - rota /usuario

- **GET** - Recuperar dados do Usuário: Back end (**express**):

```
app.get('/usuario', async (req, res) => {  
  const nomeUsuario = req.query.nome;  
  const idadeUsuario = req.query.idade;  
  try { const usuario = await buscarUsuarioPorNomeEIdade(nomeUsuario, idadeUsuario);  
    res.json(usuario);  
  } catch (error) {  
    console.error('Erro ao recuperar nome e idade do usuário:', error);  
    res.status(500).json({ error: 'Erro ao recuperar nome e idade do usuário' });  
  }  
});
```

API RESTFul - exemplo - rota /usuario

- **POST** - Criar Novo Usuário: Front end (**Fetch**):

```
const novoUsuario = {  
  nome: 'João',  
  idade: 22  
};
```

```
fetch('http://localhost:3000/usuario', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify(novoUsuario)  
})  
.then(response => response.json())  
.then(data => console.log(data))  
.catch(error => console.error('Erro ao criar usuário:', error));
```

API RESTFul - exemplo - rota /usuario

- **POST** - Criar Novo Usuário: Back end (**express**):

```
app.post('/usuario', async (req, res) => {  
  try {  
    const novoUsuario = req.body;  
    await Usuario.create(novoUsuario);  
    res.json({ message: 'Usuário criado com sucesso' });  
  } catch (error) {  
    console.error('Erro ao criar usuário:', error);  
    res.status(500).json({ error: 'Erro ao criar usuário' });  
  }  
});
```

API RESTFul - exemplo - rota /usuario

- **Patch** - Atualizar nome do usuário: Front end (**Fetch**):

```
const usuarioAtualizado = {  
  nome: 'Novo Nome',  
};
```

```
fetch('http://localhost:3000/usuario/1', {  
  method: 'PATCH',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify(usuarioAtualizado)  
})  
.then(response => response.json())  
.then(data => console.log(data))  
.catch(error => console.error('Erro ao atualizar parcialmente o usuário:', error));
```


API RESTFul - exemplo - rota /usuario

- **Patch** - Atualizar nome do usuário: Back end (**express**):

```
app.patch('/usuario/:id', async (req, res) => {  
  const idUsuario = req.params.id;  
  const dadosAtualizados = req.body;  
  try {  
    await Usuario.update(dadosAtualizados, { where: { id: idUsuario } });  
    res.json({ message: 'Usuário atualizado parcialmente com sucesso' });  
  } catch (error) {  
    console.error('Erro ao atualizar parcialmente o usuário:', error);  
    res.status(500).json({ error: 'Erro ao atualizar parcialmente o usuário' });  
  }  
});
```

API RESTFul - exemplo - rota /usuario

- **DELETE** - Remover Usuário Existente: Front end (**Fetch**):

```
fetch('http://localhost:3000/usuario/1', {  
  method: 'DELETE'  
})  
  
.then(response => response.json())  
  
.then(data => console.log(data))  
  
.catch(error => console.error('Erro ao remover usuário:', error));
```

API RESTFul - exemplo - rota /usuario

- **DELETE** - Remover Usuário Existente: Back end (**express**):

```
app.delete('/usuario/:id', async (req, res) => {  
  const idUsuario = req.params.id;  
  try {  
    await Usuario.destroy({ where: { id: idUsuario } });  
    res.json({ message: 'Usuário removido com sucesso' });  
  } catch (error) {  
    console.error('Erro ao remover usuário:', error);  
    res.status(500).json({ error: 'Erro ao remover usuário' });  
  }  
});
```

Documentação do Node e Express

- Node => página em português: <https://nodejs.org/pt-br>
- Node => documentação: <https://nodejs.org/pt-br/docs>
- Express => página em português: <https://expressjs.com/pt-br/>
- Express => instalação:
- <https://expressjs.com/pt-br/starter/installing.html>
- Node e Express => MDN
- https://developer.mozilla.org/pt-BR/docs/Learn/Server-side/Express_Nodejs/Introduction